



SWIN TRANSFORMER MİMARİSİ

CANBERK SEZGİN 1030510698

Genel Bakış

- CNN'lerin yetersizliği
- Transformer, ViT ve problemi
- Swin Transformer mimarisi
- Mimari varyantlar (T, S, B, L)
- Deneysel sonuçlar (3 görev)
- Ablation çalışması

CNN'lerin Yetersizliđi

◆ Uzun Mesafeli Bađımlılıklar

Yeterince derin olursa tüm görüntünün kapsanması.

Vanishing gradyent problemi.

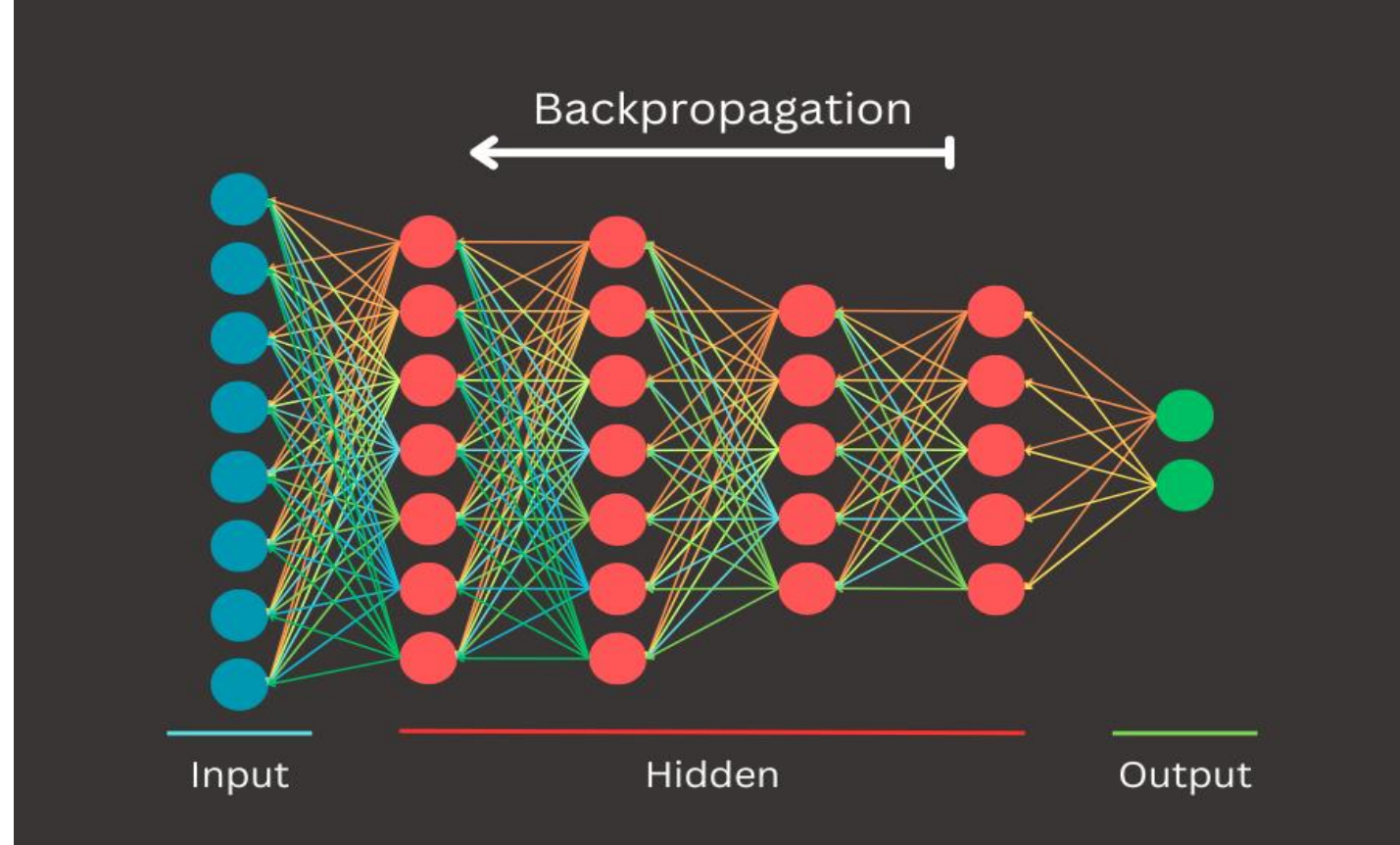
Pooling yapılarak uzamsal bilginin azalması.

◆ Ölçek Deđişkenliđi

Filtre boyutunun sabit olması, sorunları engellemek için FPN(Feature Pyramid Network) kullanıldı ama mimarının içinde olan bir çözüm deđil.

◆ Hesaplama Maliyeti

Semantik segmentasyon gibi görevlerde maliyetin çok artması.



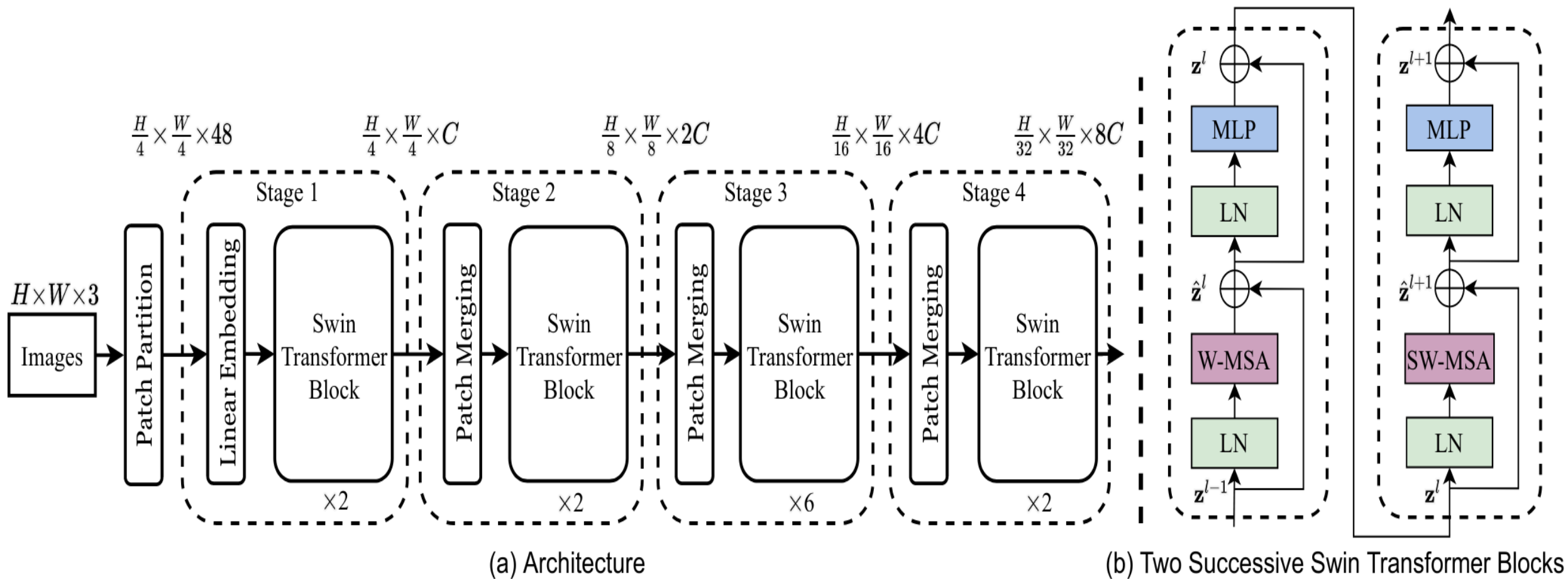
Transformer ve ViT Problemi

- ◆ Self-attention.
- ◆ Burada yeni bir problem ortaya çıkıyor karesel karmaşıklık problemi. N tane bölge varsa $O(N^2)$ işlem yapmak gerekiyor.

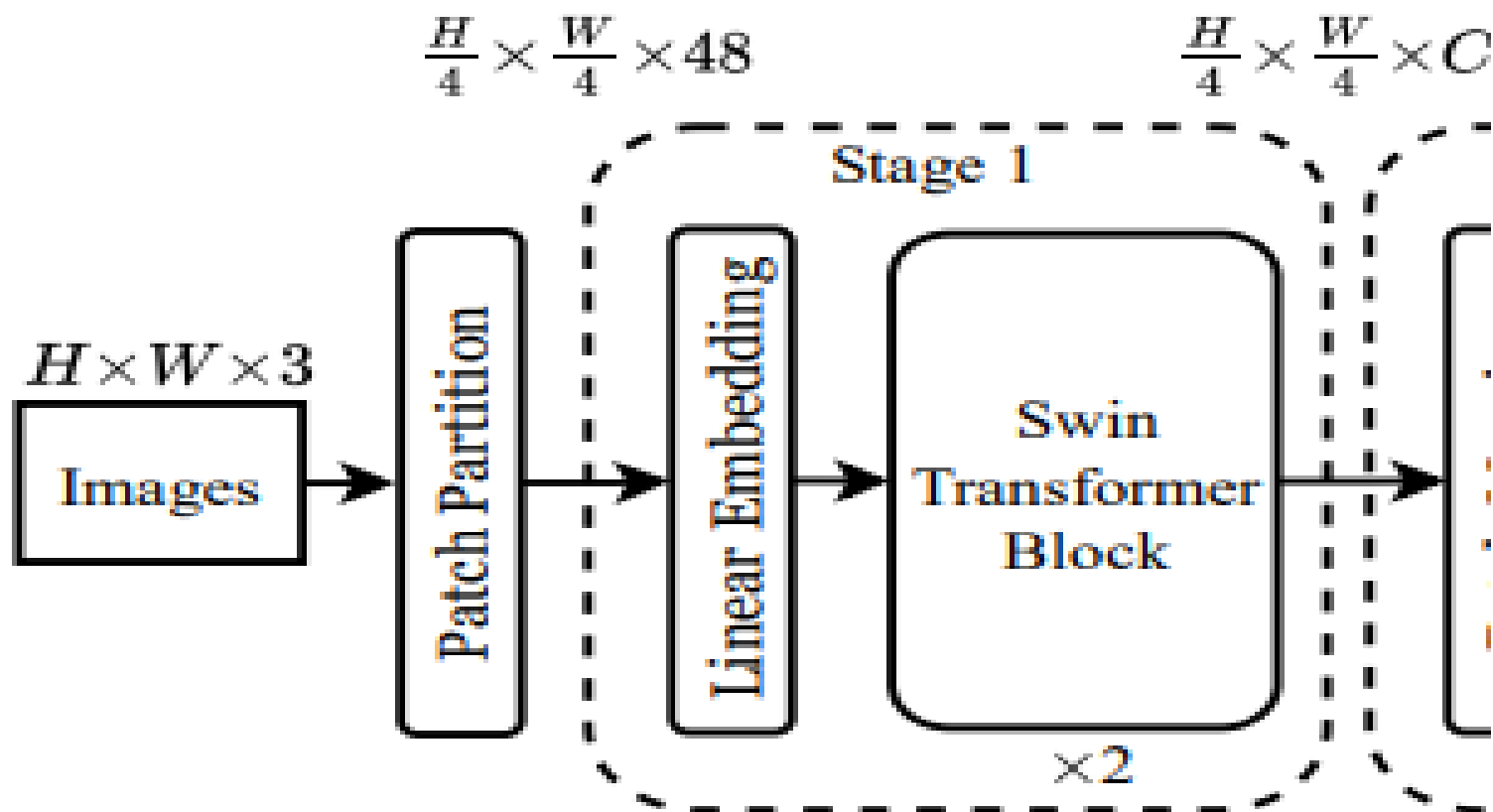
Çözüm Swin Transformer

- ◆ Hem CNN'in iyi kısımlarını hem de transformerın iyi kısımlarını aldı yeni bir mimari oluşturdu.

Swin Transformer Mimarisi



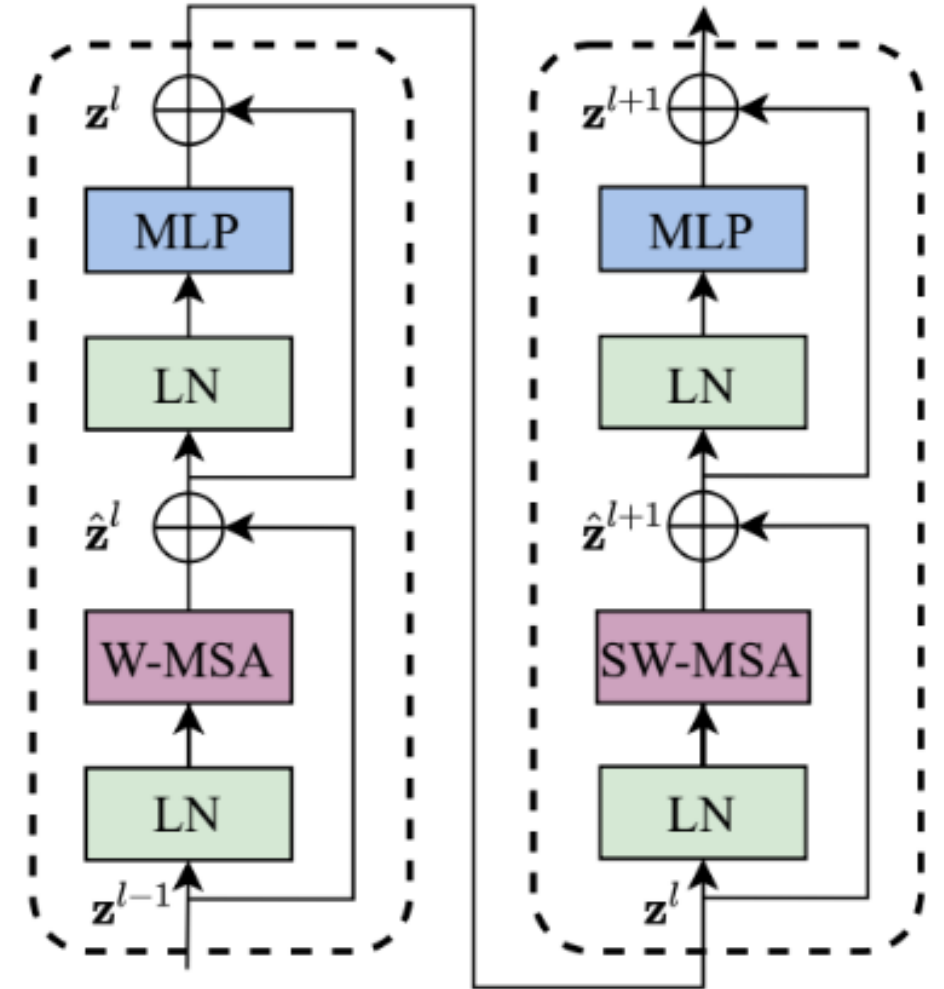
Patch Partition ve Linear Embedding



Layer Normalization

Neden Gerekli?

- ◆ Gradyan patlaması ve kayboluşu
- ◆ Öğrenme hızı hassasiyeti
- ◆ Sigmoid ve Tanh aktivasyon fonksiyonları



W-MSA

Vitte yaşadığımız karesel karmaşıklık problemini çözmeye çalışıyoruz.

Gelen yamayı 7x7 pencerelere bölüyoruz.

Yamalara attention skorarı veriyoruz (Q, K, V)

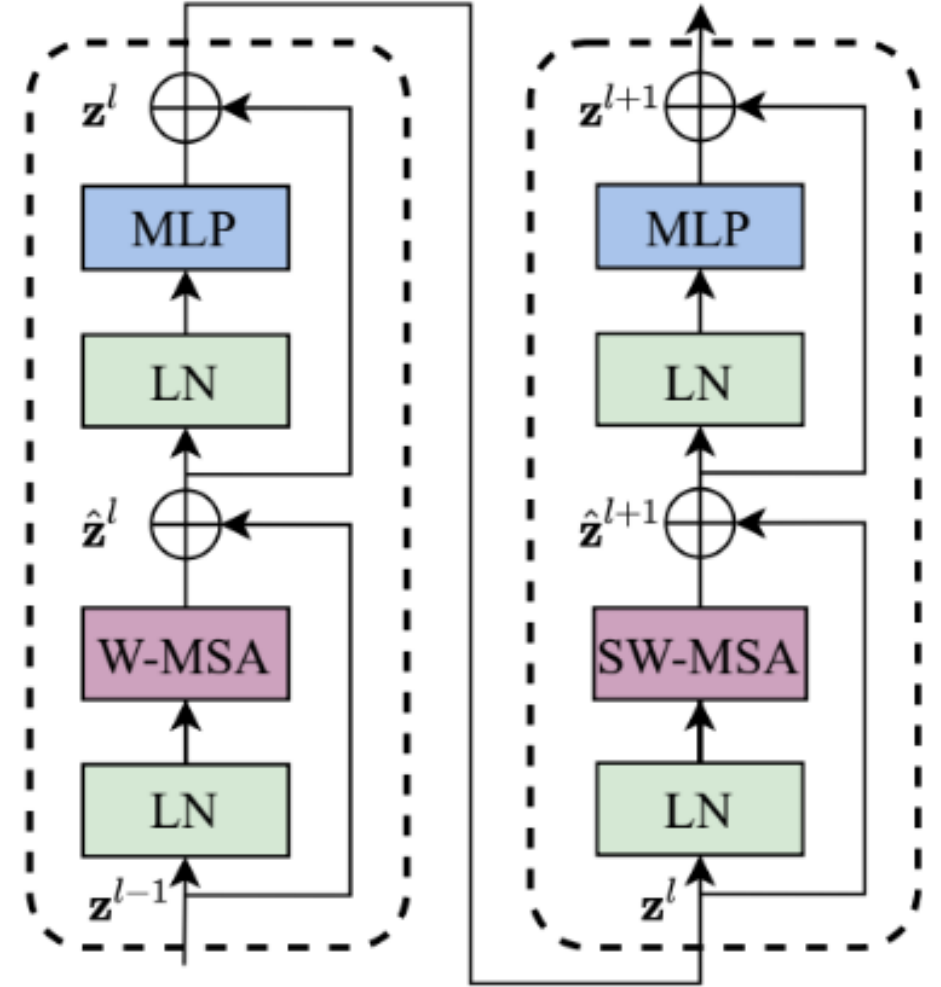
$$\text{Attention}(Q, K, V) = \text{SoftMax}(QK^T / \sqrt{d} + B) V$$

Karmaşıklık Hesabı

Görüntü boyutu 2 katına çıkarsa

ViT: $\Omega(\text{MSA}) = 4hwC^2 + 2(hw)^2C$ hesaplama 4 katına çıkar

SWiN: $\Omega(\text{W-MSA}) = 4hwC^2 + 2M^2hwC$, hesaplama 2 katına çıkar



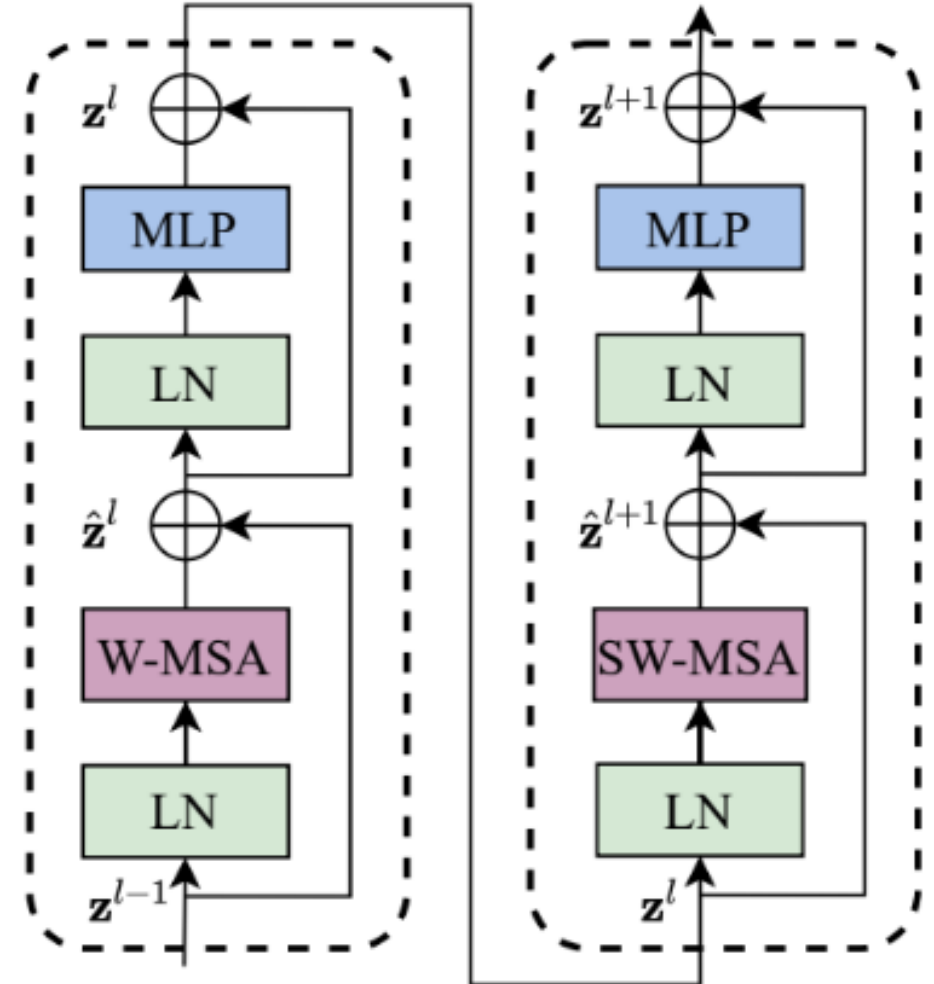
MLP (Multi Layer Perceptron)

Attention mekanizması hangi bilgiyi nerede toplayacağına karar verir MLP ise bu bilgiyle ne yapılacağına karar verir.

$$\text{MLP}(X) = W_2 \cdot \text{GELU}(W_1 \cdot X + b_1) + b_2$$

Neden aktivasyon fonksiyonları?

Neden RELU yerine GELU?

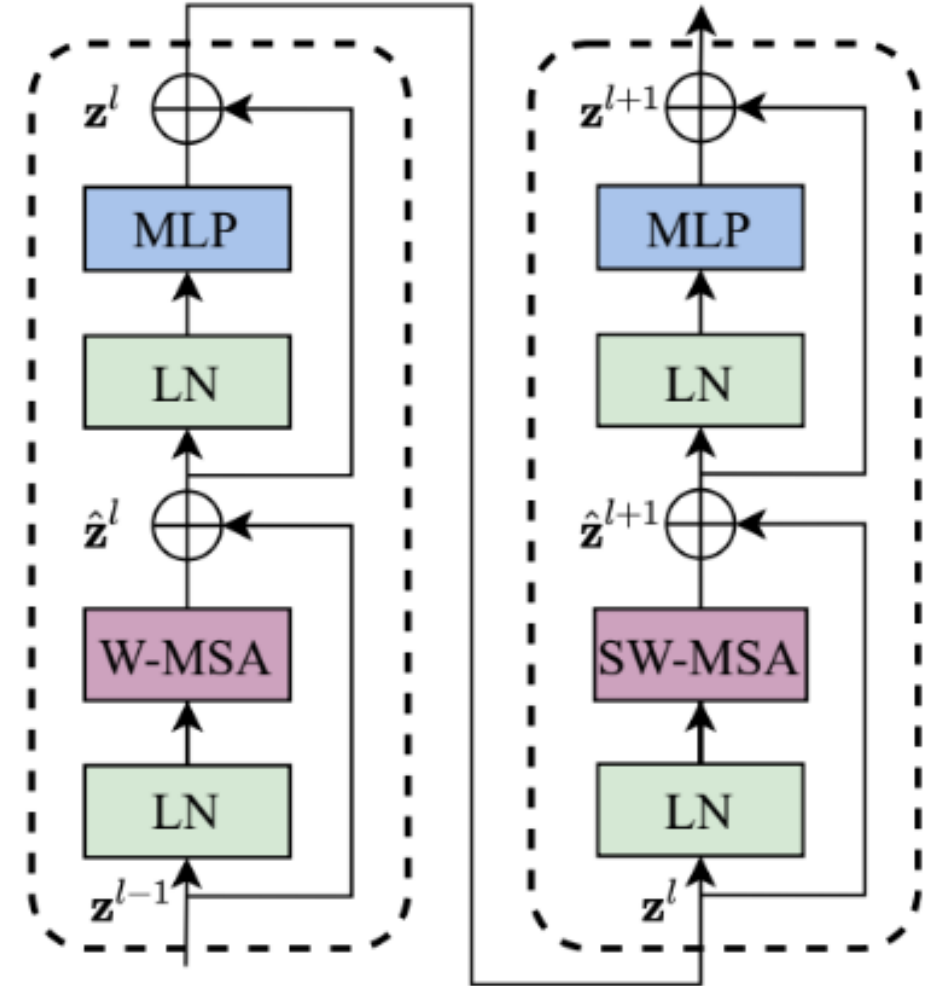


SW-MSA

0	1	2
3	4	5
6	7	8

0	1	2	
3	4	5	3
6	7	8	6
	1	2	0

$\text{Attention}(Q, K, V) = \text{SoftMax}(QK^T / \sqrt{d} + B + M)V$
 ViT absolute position embedding kullanıyordu ve her token sabit bir değer alıyordu



Patch Merging

Stage geçişlerinde kullanılır.

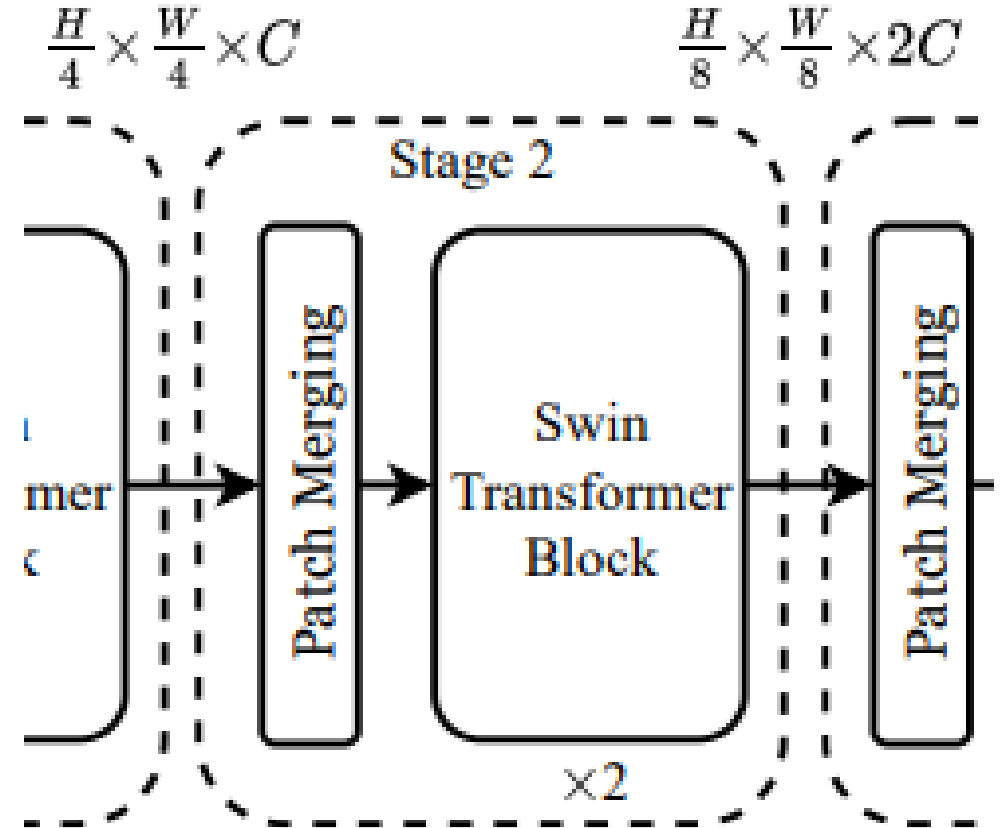
Yamadan 2x2 boyutunda bir parça alırız.

CNN max pooling ile 2x2 yama seçip en iyisini alıyordu. Swin transformerde öyle bir şey yok.

Swin Transformerda elimizdeki 2x2 yamanın her biri 96 boyutluydu toplamda 384 sayı var biz bunu ağırlık matrisleri ile yarıya indiririz yani C değeri her stagede 2 katına çıkar.

Neden çözünürlüğü azaltıyoruz?

Sonuç olarak hem çözünürlük düşer hem de geniş bölgelerin bilgilerini tek bir vektörde toplamış oluruz.



Mimari Varyantlar

Swin-T: C = 96, layer numbers = {2, 2, 6, 2}
Swin-S: C = 96, layer numbers = {2, 2, 18, 2}
Swin-B: C = 128, layer numbers = {2, 2, 18, 2}
Swin-L: C = 192, layer numbers = {2, 2, 18, 2}

Model	Parametre	FLOPs	Hız (img/s)	ImageNet Top-1
Swin Transformer Varyantları				
● Swin-T	29M	4.5G	755 img/s	81.3%
● Swin-S	50M	8.7G	437 img/s	83.0%
● Swin-B	88M	15.4G	278 img/s	83.5%
● Swin-L	197M	103.9G	42 img/s	87.3%
Karşılaştırma — Diğer Modeller				
● ViT-B/16	86M	55.4G	85.9 img/s	77.9%
● DeiT-S	22M	4.6G	940 img/s	79.8%
● DeiT-B	86M	17.5G	292 img/s	81.8%

ImageNet- Görüntü Sınıflandırma

(a) Regular ImageNet-1K trained models

method	image size	#param.	FLOPs	throughput (image / s)	ImageNet top-1 acc.
RegNetY-4G [48]	224 ²	21M	4.0G	1156.7	80.0
RegNetY-8G [48]	224 ²	39M	8.0G	591.6	81.7
RegNetY-16G [48]	224 ²	84M	16.0G	334.7	82.9
EffNet-B3 [58]	300 ²	12M	1.8G	732.1	81.6
EffNet-B4 [58]	380 ²	19M	4.2G	349.4	82.9
EffNet-B5 [58]	456 ²	30M	9.9G	169.1	83.6
EffNet-B6 [58]	528 ²	43M	19.0G	96.9	84.0
EffNet-B7 [58]	600 ²	66M	37.0G	55.1	84.3
ViT-B/16 [20]	384 ²	86M	55.4G	85.9	77.9
ViT-L/16 [20]	384 ²	307M	190.7G	27.3	76.5
DeiT-S [63]	224 ²	22M	4.6G	940.4	79.8
DeiT-B [63]	224 ²	86M	17.5G	292.3	81.8
DeiT-B [63]	384 ²	86M	55.4G	85.9	83.1
Swin-T	224 ²	29M	4.5G	755.2	81.3
Swin-S	224 ²	50M	8.7G	436.9	83.0
Swin-B	224 ²	88M	15.4G	278.1	83.5
Swin-B	384 ²	88M	47.0G	84.7	84.5

(b) ImageNet-22K pre-trained models

method	image size	#param.	FLOPs	throughput (image / s)	ImageNet top-1 acc.
R-101x3 [38]	384 ²	388M	204.6G	-	84.4
R-152x4 [38]	480 ²	937M	840.5G	-	85.4
ViT-B/16 [20]	384 ²	86M	55.4G	85.9	84.0
ViT-L/16 [20]	384 ²	307M	190.7G	27.3	85.2
Swin-B	224 ²	88M	15.4G	278.1	85.2
Swin-B	384 ²	88M	47.0G	84.7	86.4
Swin-L	384 ²	197M	103.9G	42.1	87.3

COCO-Nesne Tesbiti & Instance Segmentasyon

(a) Various frameworks

Method	Backbone	AP ^{box}	AP ^{box} ₅₀	AP ^{box} ₇₅	#param.	FLOPs	FPS
Cascade	R-50	46.3	64.3	50.5	82M	739G	18.0
Mask R-CNN	Swin-T	50.5	69.3	54.9	86M	745G	15.3
ATSS	R-50	43.5	61.9	47.0	32M	205G	28.3
	Swin-T	47.2	66.5	51.3	36M	215G	22.3
RepPointsV2	R-50	46.5	64.6	50.3	42M	274G	13.6
	Swin-T	50.0	68.5	54.2	45M	283G	12.0
Sparse R-CNN	R-50	44.5	63.4	48.2	106M	166G	21.0
	Swin-T	47.9	67.3	52.3	110M	172G	18.4

(b) Various backbones w. Cascade Mask R-CNN

	AP ^{box}	AP ^{box} ₅₀	AP ^{box} ₇₅	AP ^{mask}	AP ^{mask} ₅₀	AP ^{mask} ₇₅	param	FLOPs	FPS
DeiT-S [†]	48.0	67.2	51.7	41.4	64.2	44.3	80M	889G	10.4
R50	46.3	64.3	50.5	40.1	61.7	43.4	82M	739G	18.0
Swin-T	50.5	69.3	54.9	43.7	66.6	47.1	86M	745G	15.3
X101-32	48.1	66.5	52.4	41.6	63.9	45.2	101M	819G	12.8
Swin-S	51.8	70.4	56.3	44.7	67.9	48.5	107M	838G	12.0
X101-64	48.3	66.4	52.3	41.7	64.0	45.1	140M	972G	10.4
Swin-B	51.9	70.9	56.5	45.0	68.4	48.7	145M	982G	11.6

(c) System-level Comparison

Method	mini-val		test-dev		#param.	FLOPs
	AP ^{box}	AP ^{mask}	AP ^{box}	AP ^{mask}		
RepPointsV2* [12]	-	-	52.1	-	-	-
GCNet* [7]	51.8	44.7	52.3	45.4	-	1041G
RelationNet++* [13]	-	-	52.7	-	-	-
SpineNet-190 [21]	52.6	-	52.8	-	164M	1885G
ResNeSt-200* [78]	52.5	-	53.3	47.1	-	-
EfficientDet-D7 [59]	54.4	-	55.1	-	77M	410G
DetectoRS* [46]	-	-	55.7	48.5	-	-
YOLOv4 P7* [4]	-	-	55.8	-	-	-
Copy-paste [26]	55.9	47.2	56.0	47.4	185M	1440G
X101-64 (HTC++)	52.3	46.0	-	-	155M	1033G
Swin-B (HTC++)	56.4	49.1	-	-	160M	1043G
Swin-L (HTC++)	57.1	49.5	57.7	50.2	284M	1470G
Swin-L (HTC++)*	58.0	50.4	58.7	51.1	284M	-

ADE20K – Semantik Segmentasyon

ADE20K		val	test	#param.	FLOPs	FPS
Method	Backbone	mIoU	score			
DANet [23]	ResNet-101	45.2	-	69M	1119G	15.2
DLab.v3+ [11]	ResNet-101	44.1	-	63M	1021G	16.0
ACNet [24]	ResNet-101	45.9	38.5	-		
DNL [71]	ResNet-101	46.0	56.2	69M	1249G	14.8
OCRNet [73]	ResNet-101	45.3	56.0	56M	923G	19.3
UperNet [69]	ResNet-101	44.9	-	86M	1029G	20.1
OCRNet [73]	HRNet-w48	45.7	-	71M	664G	12.5
DLab.v3+ [11]	ResNeSt-101	46.9	55.1	66M	1051G	11.9
DLab.v3+ [11]	ResNeSt-200	48.4	-	88M	1381G	8.1
SETR [81]	T-Large [‡]	50.3	61.7	308M	-	-
UperNet	DeiT-S [†]	44.0	-	52M	1099G	16.2
UperNet	Swin-T	46.1	-	60M	945G	18.5
UperNet	Swin-S	49.3	-	81M	1038G	15.2
UperNet	Swin-B [‡]	51.6	-	121M	1841G	8.7
UperNet	Swin-L [‡]	53.5	62.8	234M	3230G	6.2

Ablation Çalışması

Yöntem	ImageNet Top-1	COCO APbox	COCO APmask	ADE20K mIoU
Shifted window yok	80.2%	47.7	41.5	43.3
Shifted window var ★ Swin	81.3% +1.1	50.5 +2.8	43.7 +2.2	46.1 +2.8

Yöntem	ImageNet Top-1	COCO APbox	COCO APmask	ADE20K mIoU
Pozisyon yok	80.1%	49.2	42.6	43.8
Mutlak pozisyon	80.5% +0.4	49.0 -0.2	42.4 -0.2	43.2 -0.6
Mutlak + Relatif	81.3%	50.2	43.4	44.0
Relatif pozisyon ★ Swin	81.3%	50.5 +1.3	43.7 +1.1	46.1 +2.3

Dinlediğiniz İçin Teşekkürler

- **Kaynakça**

<https://arxiv.org/abs/2103.14030>

<https://github.com/microsoft/Swin-Transformer>